



Java et la Programmation Orientée Objet



Houssem MAHMOUDI
houssem.mahmoudi@ensicaen.fr

Ensicaen©2025 - HM



Chapitre 3 : Gestion des exceptions

1. Introduction
2. Traitement des exceptions
3. Les exceptions prédéfinies en Java
4. Propagation/Interception d'une exception
5. Les exceptions définies par l'utilisateur
6. Exercices

Ensicaen©2025 - HM

2

I. Introduction

- En Java, il y a deux types des erreurs :
- Erreurs de syntaxe (Syntax errors)** : Erreurs commises par le programmeur, détectées pendant la phase de compilation du code Java.
 - Exemple : variable non déclarée, omission d'une parenthèse, point virgule,...
- Erreurs d'exécution (Runtime errors)** : Apparues lors de l'exécution du programme.

Exemple :

- Mauvaise entrée : division par 0, valeur négative,...
- Ressources non disponible : base de données non accessible, pas de connexion Internet.

```
public class Erreur {
    public static void main(String[] args) {
        int a = 6 ;
        double d ;
        d = a / 0 ;
        System.out.println("valeur finale : " + d);
    }
}
```

- ✓ Pas d'erreurs de compilation
- ✓ Erreur d'exécution

I. Introduction

- Le programme compile normalement, mais lors de l'exécution on obtient une erreur :

```
C:\myjava\exception>javac TestErreur.java
C:\myjava\exception>java TestErreur
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at TestErreur.main(TestErreur.java:6)
C:\myjava\exception>
```

Compilation : OK

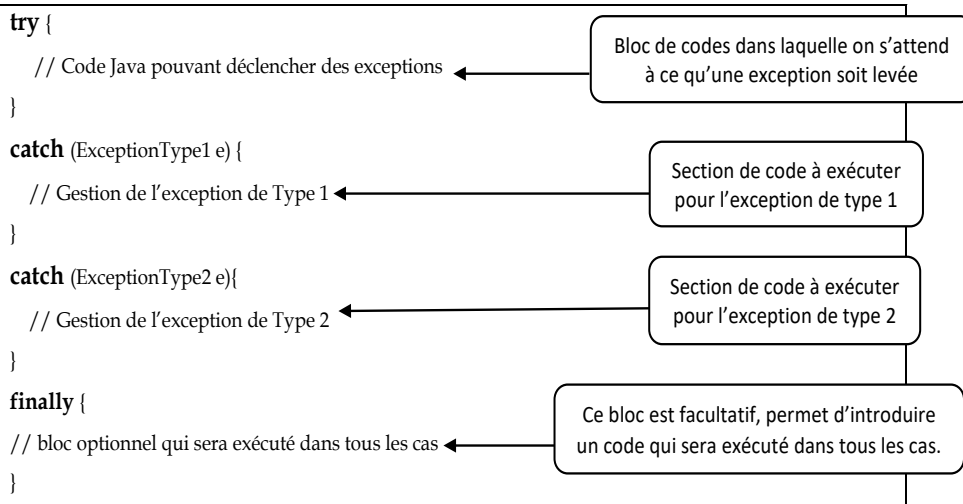
Exécution : erreur

- Les erreurs d'exécution, appelée aussi des **exceptions**, ne sont pas traitées par le compilateur, c'est le travail du programmeur qui doit deviner et détecter ces erreurs et les traiter convenablement.

II. Traitement des exceptions

- En Java, une **exception** est un objet représentant une erreur ou une situation imprévue qui se produit pendant l'exécution d'un programme.
- La **gestion des exceptions** est un mécanisme permettant de gérer les erreurs sans que le programme se crache.
- En Java, le traitement des exceptions se fait par :
 - **try** : Définit un bloc de codes où une exception peut se produire.
 - **catch** : Capture et traite une exception.
 - **finally** : Bloc de codes qui sera exécuté dans tous les cas (exception levée ou non).
 - **throw** : Permet de lever une exception.
 - **throws** : Déclare qu'une méthode peut potentiellement lever une exception.

II. Traitement des exceptions



II. Traitement des exceptions

Exemple 1 :

- Lorsque `res = A[0] / A[1]` est exécuté, il déclenche une `"ArithmeticException"` (division par zéro).
- L'accès à `A[6]` n'est jamais atteint car l'exception coupe l'exécution du bloc `try`.
- Une seule exception sera interceptée et le message "Division par zéro" sera exécuté.
- Dans un bloc `catch`, la première exception ne peut pas être la super classe des autres exceptions suivantes. (masque du traitement).

```
public static void main(String[] args) {
    int A[] = {10,0,7,6,9};
    int res;
    try
    {
        res = A[0]/A[1];
        System.out.println(res);
        System.out.println(A[6]);
    }
    catch(ArrayIndexOutOfBoundsException e) {
        System.out.println("indice non valide");
    }
    catch(ArithmeticException e) {
        System.out.println("Division par zéro!!");
    }
}
```

II. Traitement des exceptions

Exemple 2 :

- Il s'agit d'un bloc de `try..catch` imbriqué.
- La 1^{ère} `catch` attrape l'exception `ArithmeticException`, son bloc sera exécuté.
- Le programme continu à s'exécuter, une autre exception sera levée de type `ArrayIndexOutOfBoundsException` qui sera attrapée par la 2^{ème} `catch`.

```
public static void main(String[] args) {
    int A[] = {10,0,7,6,9};
    int res;
    try
    {
        try
        {
            res = A[0]/A[1];
            System.out.println(res);
        }catch(ArithmeticException e) {
            System.out.println("Division par zéro");
        }
        System.out.println(A[6]);
    }catch(ArrayIndexOutOfBoundsException e) {
        System.out.println("indice non valide");
    }
}
```

II. Traitement des exceptions

Exemple 3 :

- L'exception **NullPointerException** est levée si le programme tente d'appeler une méthode sur un objet null.
- Le bloc **finally** est optionnel, il sera toujours exécuter dans toutes les situations (exception levée ou non).

```
public static void main(String[] args) {
    String texte = null;
    try
    {
        System.out.println(texte.length());
    }
    catch (NullPointerException e)
    {
        System.out.println("Accès à un objet null!");
    }
    finally
    {
        System.out.println("Programme continu...");
    }
}
```

III. Les exceptions prédéfinies en Java

- En Java les exceptions sont des objets de type **Exception**, tous ces objets héritent de la classe **Throwable**.
- La classe Throwable est la classe mère de toutes les **exceptions** et les **erreurs**.
 - **Exception** : Représente les exceptions que l'on peut gérer dans un programme.
 - **Error** : Représente les erreurs graves dû aux matériels ou système d'exploitation (utilisé par le JVM). Ce genre des erreurs sont rares et ne sont pas traités par programmation.
- On distingue deux types des exceptions :
 - **Checked Exceptions** : elles doivent obligatoirement être traitées avec un bloc try-catch (vérifiés par le compilateur) ou déclarées avec **throws**.

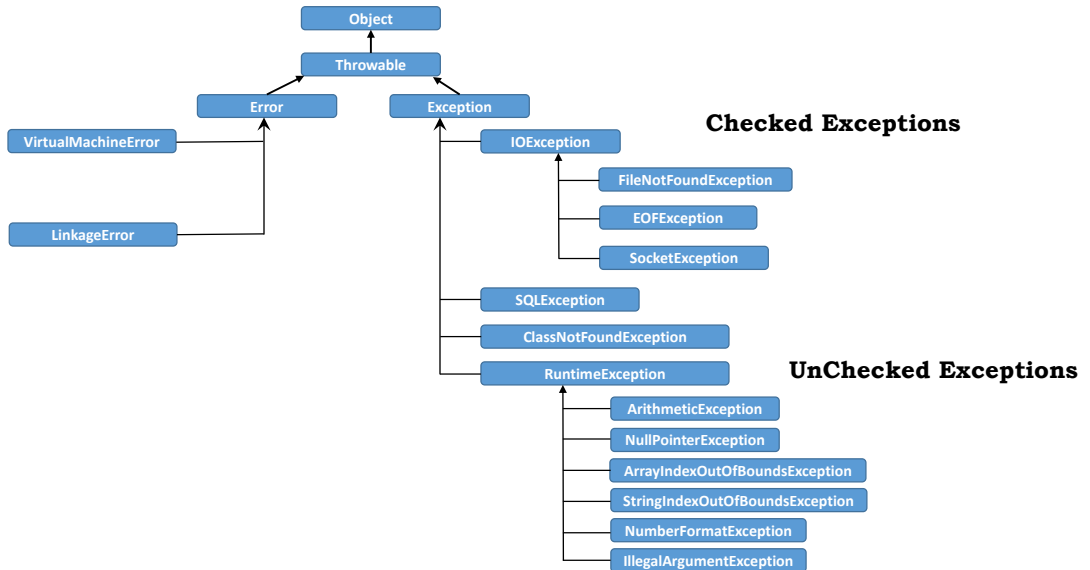
Exemples : IOException, SQLException, FileNotFoundException

- **Unchecked Exceptions** : elles héritent de **RuntimeException** et ne nécessitent pas de traitement obligatoire.

Exemples : NullPointerException, ArrayIndexOutOfBoundsException.

III. Les exceptions prédéfinies en Java

- Hiérarchie des exceptions, non exhaustive :



III. Les exceptions prédéfinies en Java

- **ArithmeticException** : Exception levée en cas d'une erreur lors d'opération arithmétique.
- **FileNotFoundException** : Exception levée lors d'un accès à un fichier qui n'existe pas dans le disque.
- **NullPointerException** : Exception levée lors d'un accès à un objet null.
- **IOException** : Exception levée lorsqu'une erreur d'entrée/sortie se déclenche.
- **IndexOutOfBoundsException** : Exception est levée lorsqu'on tente d'accéder à une position invalide dans un tableau / une chaîne de caractère. C'est la classe de base de *StringIndexOutOfBoundsException* et *ArrayIndexOutOfBoundsException*
- **NumberFormatException** : Exception est levée lors de la conversion d'une chaîne de caractères en nombre.
- **ClassNotFoundException** : Exception est levée en cas de problème de chargement de classe.
- Pour intercepter toutes les exceptions il suffit d'écrire **catch (Exception e)**.

IV. Propagation des exceptions entre les méthodes

- Si une méthode lève une exception, alors il faut soit :
 - **Interceptor** et traiter l'exception try .. catch.
 - **Propager** l'exception vers la méthode appelante.
- Pour propager une exception on doit ajouter après l'entête de la méthode le mot clé **throws** suivi du type d'exception qu'elle doit propager. La méthode appelante doit traiter l'exception.

IV. Propagation des exceptions entre les méthodes

- Cas d'interception :

```
public int ajouter(int a, String str) {
    try {
        int b = Integer.parseInt(str);
        a = a + b;
    } catch (NumberFormatException e) {
        System.out.println(e.getMessage());
    }
    return a;
}
```

- Cas de **propagation** :

```
public int ajouter(int a, String str) throws NumberFormatException {
    int b = Integer.parseInt(str);
    a = a + b;
    return a;
}
```

- La méthode qui a appelée ajouter() doit impérativement traiter l'exception dans un bloc try/catch

IV. Propagation des exceptions entre les méthodes

▪ Exemple

```
public class TestPropagation {
    static void meth1() throws ArithmeticException {
        meth2();
    }
    static void meth2() throws ArithmeticException {
        meth3();
    }
    static void meth3() throws ArithmeticException {
        int a=5,b=0;
        System.out.println(a/b);
    }
    public static void main(String[] args) {
        try {
            meth1();
        } catch (ArithmeticException e) {
            System.out.println("Erreur : " + e.getMessage());
        }
    }
}
```



➤ Dans ce cas, la fonction **main** doit traiter l'exception dans un bloc try .. catch

V. Les exceptions définies par l'utilisateur

- Java offre la possibilité au programmeur de créer ses propres exceptions.
- Les exceptions définies doivent hériter de la classe **Exception** (ou l'une de ses dérivées).
- Le programmeur doit lui-même lever ses propres exception par le mot clé **throw**.
- Le traitement de l'exception reste avec le même mécanisme (try .. catch .. finally)
- On peut redéfinir les méthodes suivantes héritées de la classe Exception :
 - String getMessage()
 - void printStackTrace()
 - String toString()

Exercice :

- Ecrire une classe principale qui permet de calculer la surface d'un rectangle. Le programme doit lever une exception en cas d'enter une dimension négative.

V. Les exceptions définies par l'utilisateur

```
class MyException extends Exception {
    public String getMessage() {
        return "Dimension négative.";
    }
}

public class TestExceptionPredefinie {
    static double surface(double a, double b) throws MyException{
        if(a<0 || b<0)
            throw new MyException();
        return a*b;
    }

    static void verifier() {
        try {
            System.out.println("Surface = " + surface(-7,5));
        } catch (MyException e) {
            System.out.println("Erreur... " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("x = "); double x = sc.nextDouble();
        System.out.print("y = "); double y = sc.nextDouble();
        verifier(x,y);
    }
}
```

- Définition d'une classe qui hérite de Exception.
- Overriding de la méthode getMessage()

- Propager le traitement de l'exception vers une autre méthode.

- Déclenchement de MyException.

- Traitement habituel de l'exception.

Exercice 1

Analyser le programme et déterminez le résultat final

```
public class DemoClass {
    public static void main(String[] args) {
        int k=0;
        try {
            int i = 5/k;
            System.out.print("0");
        } catch (ArithmeticException e) {
            System.out.print("1");
        } catch (RuntimeException e) {
            System.out.print("2");
            return;
        } catch (Exception e) {
            System.out.print("3");
        } finally {
            System.out.print("4");
        }
        System.out.print("5");
    }
}
```

Exercice 2

Analyser le programme et apporter les modifications nécessaires pour qu'il fonctionne

```
class A extends Throwable {}
class B extends A {}
public class Extra {
    public static void main(String[] args) throws A {
        try {
            action();
        } catch (A e) {
            System.out.println("Ok");
        } catch (B e) {
            System.out.println("OK2");
        } finally {
            System.out.println("Done.");
        }
    }
    static void action() throws B {
        throw new B();
    }
}
```

Exercice 3

Analysez le programme et déduisez le résultat final.

```
public class DriveBus {
    public void go() {
        System.out.print("A");
        try {
            stop();
            System.out.print("B");
        } catch (ArithmeticException e) {
            System.out.print("C");
        } catch (IndexOutOfBoundsException e) {
            System.out.print("D");
        } catch (Exception e) {
            System.out.print("E");
        } finally {
            System.out.print("F");
        }
        System.out.print("G");
    }
}
```

```
public void stop() {
    System.out.print("H");
    Object x = null;
    x.toString();
    System.out.print("I");
}
public static void main(String pm[]) {
    new DriveBus().go();
    System.out.print("J");
}
```

Exercice 4

Analyser le programme et déterminez le résultat final

```
public class PropagationException {
    public static void methodA() throws Exception {
        try {
            System.out.print("A1 ");
            methodB();
            System.out.print("A2 ");
        } catch (NullPointerException e) {
            System.out.print("A3 ");
        } catch (ArithmeticException e) {
            System.out.print("A5 ");
        } catch (RuntimeException e) {
            System.out.print("A4 ");
        } finally {
            System.out.print("A6 ");
        }
    }
}
```

```
public static void methodB() throws Exception {
    try {
        System.out.print("B1 ");
        throw new Exception("Exception in B");
    } finally {
        System.out.print("B2 ");
    }
}

public static void main(String[] args) {
    try {
        methodA();
    } catch (RuntimeException e) {
        System.out.print("Main ");
    } catch (Exception e) {
        System.out.print("Main2 ");
    }
}
}
```

Exercice 5

- Définir une classe « **Division** » qui contient une méthode `diviser(int a, int b)`.
- Lever une exception définie « **Stop** » si le diviseur est 0.
- La classe `Stop` doit redéfinir la méthode `getMessage()` pour retourner un message convenable.
- Ecrire le code de la classe principale « `CalculDivision` » qui appelle la méthode `diviser`.

Exercice 6

- On vous demande de réaliser une classe **EntierNat** permettant de manipuler des entiers naturels (positifs ou nuls).
- Cette classe dispose :
 - D'un attribut privé de type entier.
 - D'un constructeur à un argument pour initialiser l'attribut, il générera une exception de type **ErreurEntier** (exception à définir) lorsque la valeur reçue ne conviendra pas.
 - Dans l'exception à définir, il est nécessaire de redéfinir la méthode getMessage().
 - D'une méthode getN () permet de retourner la valeur de l'attribut.
- Écrire un programme principal qui teste l'utilisation de la classe EntierNat et gère l'exception "ErreurEntier". Si l'exception est levée, afficher le message nécessaire.